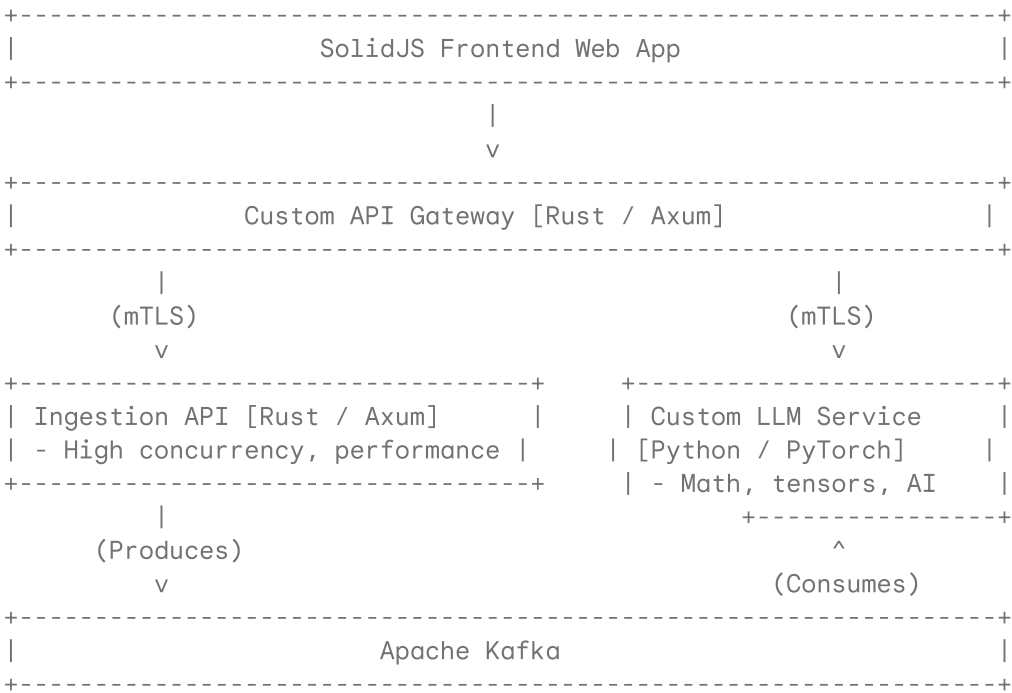


Project Millipede: Polyglot Architecture & Stack Design

This document details the microservices architecture, data layers, and coding language distribution for the DevSecOps Team Radar.

The Polyglot Language Matrix

In modern cloud environments, selecting the optimal language for each subsystem is highly effective. For `millipede`, the workload is split between **Rust** and **Python**:



1. Where We Use Rust (The Heavy Lifters)

Rust's compile-time guarantees, deterministic memory allocation, and concurrency performance are ideal for the core infrastructure pipelines:

- **The Ingestion API:** Handles concurrent webhooks from Git, Slack, and Jira using the `Axum` framework.
- **The Custom API Gateway:** Manages high-frequency reverse-proxy tasks and token signatures.
- **The Core Analyzer:** Acts as the final step in the pipeline. It consumes enriched data from Kafka, queries PostgreSQL, and updates Redis keys using the `async SQLx crate`.

2. Where We Use Python (The Deep Learning Domain)

While deep learning can be compiled into binary runtimes, Python's mathematical libraries are perfect for training and model definition:

- **The Custom LLM Worker:** Implements tokenizers, matrix self-attention modules, and model weights using native PyTorch, listening to incoming raw stream topics directly.

The Real-Time Data Plane

- **Apache Kafka:** Serves as the immutable, partitioned transaction ledger connecting our ingestion, analysis, and processing phases.
- **PostgreSQL:** Serves as our relational database. It maintains historical timelines, team mapping hashes, resolved metrics, and security audits.
- **Redis:** Powers real-time dashboard reactivity. It caches current metrics for fast page loads and uses Redis Pub/Sub to send events directly to your dashboard's active sockets.